

Introdução ao *Backend*



BACK END DEVELOPER

O que é o backend?



O que é o backend?

- Processamento de dados
- Regras de negócio
- Acesso ao banco de dados
- Segurança, autenticação e permissões
- Geração de relatórios

Tudo que não é visível faz parte do backend

Exemplo: O que o backend faz quando compro algo?



Entendendo algoritmos

Um guia ilustrado para programadores e outros curiosos

Aditya Y. Bhargava

novatec

MANNING

VIDEO

Ler amostra

Seguir o autor

Aditya Bhargava

Seguir

Entendendo Algoritmos

Capa comum – 24 abril 2017

Edition Português | por **Aditya Y. Bhargava** (Autor)

4,8      (5,048)

1º mais vendido em Programação de Computadores

[Ver todos os formatos e edições](#)

ou R\$ 57,00 em até 2x de R\$ 28,50 sem juros [Ver parcelas disponíveis](#) ▼

Parcela em até 15x sem juros com o cartão Amazon. Peça já o seu!

Um guia ilustrado para programadores e outros curiosos. Um algoritmo nada mais é do que um procedimento passo a passo para a resolução de um problema. Os algoritmos que você mais utilizará como um programador já foram descobertos, testados e provados. Se você quer entendê-los, mas se recusa a estudar páginas e mais páginas de provas, este é o livro certo. Este guia cativante e completamente ilustrado torna simples aprender como utilizar os principais algoritmos nos seus programas. O livro Entendendo Algoritmos apresenta uma abordagem agradável para esse tópico essencial da ciência da computação. Nele, você aprenderá como aplicar algoritmos comuns nos problemas de programação enfrentados diariamente. Você começará com tarefas básicas como a ordenação e a pesquisa. Com a prática, você enfrentará problemas mais complexos, como a compressão de dados e a inteligência artificial. Cada exemplo é apresentado em detalhes e inclui diagramas e códigos completos em Python. Ao final deste livro, você terá dominado algoritmos amplamente aplicáveis e saberá quando e onde utilizá-los. O que este livro inclui A abordagem de algoritmos de pesquisa, ordenação e algoritmos gráficos Mais de 400 imagens com descrições detalhadas Comparações de desempenho entre algoritmos Exemplos de código em Python Este livro de fácil leitura e repleto de imagens é destinado a programadores autodidatas, engenheiros ou pessoas que gostariam de recordar o assunto.

[Relatar um problema com este produto](#)

Número de páginas	Idioma	Editora	Data da publicação
 264 páginas	 Português	 Novatec	 24 abril 2017

Disponível instantaneamente

prime

Outros UsadoNovo, Colecionador de R\$ 48,00 ▼

R\$ 54¹⁵

De: R\$75,00 

à vista no Pix

Ou em até 2x de R\$ 28,50/mês

prime

Entrega GRÁTIS: **quarta-feira, 30 de julho.**
Se pedir dentro de **8 hrs 30 mins**

 Enviar para Ângelo - Dois Vizinhos 85660000

Em estoque

Quantidade: 1 ▼

[Adicionar ao carrinho](#)

[Comprar agora](#)

Enviado por	Amazon.com.br
Vendido por	Amazon.com.br
Devolução	Elegível para Devolução, Reembolso em até 30 dias após o recebimento
Pagamento	Transação segura

- Verifica se o usuário está logado
- Verifica se o produto está disponível
- Verifica se o cartão informado é válido
- Registra a compra no banco de dados
- Gera um número de pedido
- Responder: “Compra realizada com sucesso” (Ou bloquear a compra)

Além de:

- Calcular promoções
- Calcular formas de parcelamento disponíveis
- Recuperar endereço informado pelo cliente na última compra
- Verificar se o membro é prime, no caso da *Amazon*
- etc etc etc

Quem, o que come, onde vive o desenvolvedor backend?

Frontend



Beckend



Responsabilidades

- Criar e manter APIs
- Conectar e manipular o banco de dados
- Garantir a performance e escalabilidade
- Proteger os dados (autenticação, validação, criptografia)
- Criar testes automatizados
- Documentar o backend para que o frontend consiga consumir

Perfil

- Pensamento crítico e analítico
- Trabalho em equipe
- Curioso
- Organizado e focado em boas práticas
- Preocupado com performance e escalabilidade
- Gostar de testes e automações em geral

Qual o básico de conhecimento de um dev backend?

“Uma pessoa que sabe programar é poderosa. Uma pessoa que sabe programar e sabe como o negócio funciona é imparável.”

14 Hábitos de Desenvolvedores Altamente Produtivos

Dominar uma linguagem de programação
backend

Saber construir e consumir APIs

Banco de dados - SQL e NoSQL

Autenticação - JWT, OAuth2

Versionamento - Git e conventional commits

Testes - Unitários, integração

Segurança - Validação de entradas, proteção
contra ataques, OWASP top 10

Ambiente - Noção básica de deploy, docker,
CI/CD

Documentação - README, OpenAPI, Swagger,
Postman

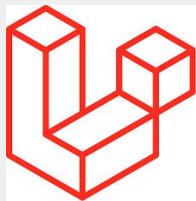
Oportunidades

- Software Houses (CISS, Precisa, SS, ML, Magalu)
- Bancos e *fintechs* (Itaú, Stone, Cresol)
- E-Commerces
- Startups
- Freelance

E qual a melhor linguagem?



express



django



A pergunta correta é: “Qual problema eu preciso resolver?”

Bora conhecer um pouco das linguagens de programação usadas no backend!

Objetivo

- Conhecer a existência de diferentes linguagens de programação utilizadas para o desenvolvimento backend.
- Entender as vantagens e desvantagens de cada uma delas.
- Entender onde cada uma pode se sair melhor

Linguagens que vamos conhecer

- NodeJs (JavaScript)
- PHP
- Python
- Java
- C#
- C++
- Ruby
- Go
- Rust



Rust

Principais utilizações

- Sistemas de alto desempenho e segurança;
- Sistemas críticos

Vantagens

- Segurança de memória
- Excelente performance
- Moderna e inteligente

Desvantagens

- Curva de aprendizado elevada (difícil)
- Ainda em amadurecimento
- Desenvolvimento mais lento (mais rigoroso)





C#

Principais utilizações

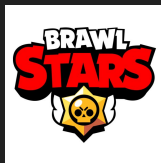
- Aplicações corporativas
- Jogos
- APIs

Vantagens

- Excelente integração com Windows e Azure
- Excelente integração com banco de dados SQL Server
- Ferramentas poderosas (Visual Studio)
- Boa performance

Desvantagens

- Historicamente mais associada ao Windows
- Menor comunidade fora do Windows





C++

Principais utilizações

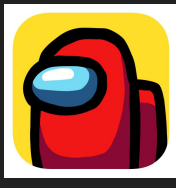
- Jogos
- Sistemas embarcados
- Banco de dados
- Motor gráficos

Vantagens

- Alto desempenho
- Controle total da memória*
- Adequado para sistemas críticos

Desvantagens

- Complexo
- Curva de aprendizado alta (difícil)
- Mais propenso a erros
- Ciclo de desenvolvimento mais lento





Ruby

Principais utilizações

- Desenvolvimento Web
- MVPs rápidos

Vantagens

- Alta produtividade (com Rails)
- Código limpo e elegante
- Boa para prototipação e startups

Desvantagens

- Performance inferior
- Dificuldade em escalar
- Comunidade em declínio



GoLang

Principais utilizações

- APIs
- Micro-serviços
- Ferramentas CLI
- Cloud

Vantagens

- Sintaxe simples
- Curva de aprendizado menor (mais fácil)
- Excelente performance
- Concorrência nativa (go routines)
- Compilado com binários pequenos

Desvantagens

- Falta de recursos mais avançados
- Gestão de dependência evolui devagar
- Poucas palavras reservadas (exige muita implementação do zero)





PHP

Principais utilizações

- Web
- Ecommerces

Vantagens

- Fácil de aprender e bem documentado
- Grande comunidade
- Suporte nativo em quase todos os servidores web

Desvantagens

- Histórico de más práticas*
- Menor desempenho
- Inconsistência na linguagem (nome de funções, parâmetros, etc)





Python

Principais utilizações

- APIs
- Automações
- Ciência de dados
- IA

Vantagens

- Sintaxe simples e legível
- Ecossistema muito rico
- Alta produtividade

Desvantagens

- Performance inferior (interpretado)
- Multithreading limitado
- Menos adequado para aplicações altamente concorrentes



angelohank / recomendacao-curso-GA



angelohank / merge-data-analyzer





Java

Principais utilizações

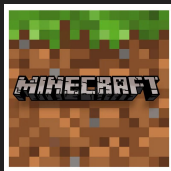
- Sistemas corporativos
- Aplicações financeiras
- APIs
- Android

Vantagens

- Extremamente robusta e escalável
- Grande ecossistema de bibliotecas e frameworks
- Amplo suporte corporativo
- Garbage Collector

Desvantagens

- Verbosidade
- Curva de aprendizado maior
- Consumo de memória alto (pesado)





Node

Principais utilizações

- APIs Rest
- Apps de tempo real (websocket, chats, etc)
- Micro-serviços

Vantagens

- Alta performance com I/O assíncrono
- Ecossistema grande via npm
- Ideal para aplicações *real time*
- Permite usar a mesma linguagem no frontend e backend

Desvantagens

- Callback hell
- Performance inferior em CPU-bound
- Maturidade menor em comparação com outras linguagens



Hello World!

 hello-cpp.cpp

 hello-csharp.cs

 hello-go.go

 hello-java.java

 hello-node.js

 hello-php.php

 hello-python.py

 hello-ruby.rb

 hello-rust.rs

Conceitos comuns para um desenvolvedor backend

API

Application Programming Interface



SERVIDOR

Um computador que nunca dorme, que contém arquivos, recursos, etc

Servidor X API

Não são necessariamente coisas diferentes, mas sim **camadas** diferentes da aplicação.

E uma aplicação simples, eles “vivem” juntos na mesma máquina.

Rotas

*Caminho usado para acessar um **recurso***

```
/recados -> leva aos recados  
/usuarios -> leva aos usuários  
/produtos -> leva aos produtos
```

```
/ -> rota raiz
```

MÉTODO
CAMINHO
FUNÇÃO



REQ (Requisição)

Pedido que o cliente (navegador) faz

- O que quer? (rota)
- Como quer? (método http)
- Que informações quer passar?
 - params, body, query



RES (Resposta)

Retorno do lado do servidor

- Prato pronto
- Mensagem de que foi pedido
- Sempre devolve um status



JSON

JavaScript Object Notation

example.json

x

```
{
  "nome": "Lucas",
  "idade": 19,
  "esta_matriculado": true,
  "curso": "Aprender E Crescer",
  "linguagens": ["Java", "Python", "JavaScript"]
}
```

